

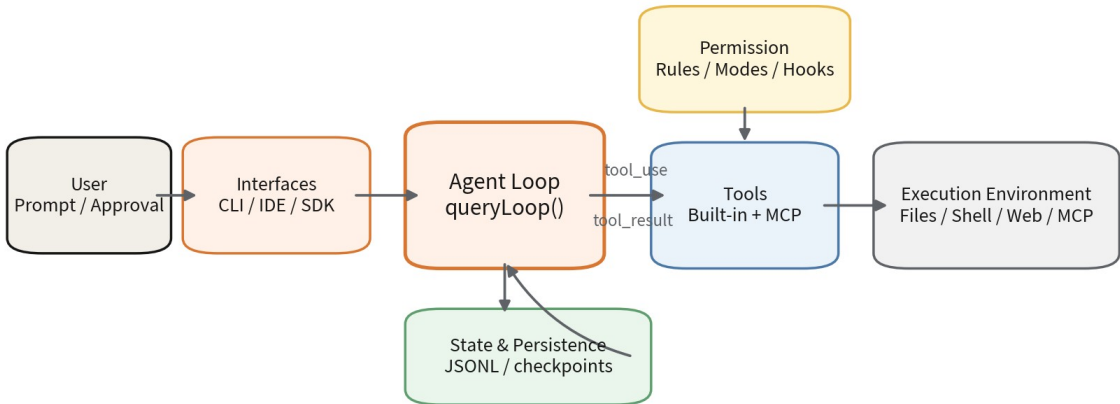
CLAUDE CODE

源码架构深度分析

基于 2026-03-31 Source Map 事件、公开学术源码分析与官方文档的工程解构

Agent Loop • Tool Runtime • Permission • Sandbox • Context • Memory
Skills • Hooks • MCP • Subagents • Worktrees • Persistence

Claude Code：七组件高层结构（本报告重绘）



Research cutoff: 2026-08-13 | Chinese Engineering Edition

如何阅读这份报告

范围与伦理边界 本报告不提供、复制或链接未授权的专有源码镜像，也不复现大段泄露代码。分析对象是公开确认的事件、公开学术源码分析、Claude Code 官方文档以及主流媒体对“源码快照中发现内容”的报道。所有代码片段均为本报告原创伪代码。

这份报告的目标不是“猎奇看泄露”，而是回答三个工程问题：

- Claude Code 的真正系统中心是什么？
- 为什么一个看似简单的 Agent Loop 会长成几十万行生产代码？
- 哪些设计可以迁移到自己的 Coding Agent / Agent Runtime？

核心结论 Claude Code 的秘密不在一个神奇 Prompt，也不在一个复杂 planner。最值得学习的是：极简 reactive loop 之上，堆叠了 context governance、tool execution、permission/sandbox、recovery、extensibility、delegation 和 persistence。

证据分级

等级	来源类型	可用于什么结论	本报告处理方式
A	Anthropic 官方声明 / Claude Code Docs	当前公开功能、权限语义、合规与产品状态	作为事实基线
B	2026-07 v2 学术源码分析 (arXiv:2604.14228)	v2.1.88 快照的模块边界、文件映射、实现模式	作为源码级主要证据；同时保留作者的证据等级提醒
C	WSJ / Ars / The Verge / BleepingComputer / DeepLearning.AI 等	事件经过、未发布功能的多源报道	用于路线图线索，不视作产品承诺
D	社区博客 / 逆向项目	发现线索、术语、经验性观察	仅辅助，不作为关键架构结论

“泄露发现”应按证据强度阅读，而不是按社交媒体热度阅读

打包错误 + 无客户凭据暴露	官方声明	高	可以作为事实
queryLoop / permission / compaction / module map	学术源码分析 + 官方功能交叉验证	高-中	适合工程学习
KAIROS / AutoDream / Buddy	多家主流媒体基于源码快照报道	中	视作路线图线索
Undercover 等内部模式	媒体 + 社区解读	中-低	不要等同于产品承诺
未来模型代号/具体发布日期	社区传播	低	不纳入架构结论

图 0-1 证据强度矩阵：先判断“来源强度”，再判断“架构价值”。

最值得先看的章节

如果你的目标是…	优先阅读
理解 Claude Code 本体	第 3-10 章：Loop → Tool → Permission → Context → Extension → Subagent → Persistence
自己造一个 Mini Claude Code	第 12、14、15 章
研究 Context Engineering	第 7 章
研究 Agent 安全	第 6 章
研究 Multi-Agent	第 9 章
研究源码泄露本身	第 1、2、13 章

目录

1. 事件取证：到底泄露了什么？
 2. 先纠正五个常见误解
 3. 源码地图：从 50 万行看到 7 个系统组件
 4. 核心：Agentic Query Loop
 5. Tool Runtime：并发、流式与结果回写
 6. Permission & Sandbox：真正的安全边界
 7. Context & Memory：Claude Code 的第一资源约束
 8. Extensibility：MCP / Plugins / Skills / Hooks
 9. Subagents / Worktrees / Agent Teams
 10. Session Persistence / Compact / Resume / Rewind
 11. 隐藏功能与路线图线索：什么能信，什么不能信
 12. 从源码抽象出的 13 个设计原则
 13. 泄露事件本身暴露的工程问题
 14. 复刻一个 Mini Claude Code：最小可行 Runtime
 15. 从 s01 → s10：可累积的 Agent Runtime 教学路线
 16. 源码阅读方法：如何分析一个 50 万行 Agent 产品
 17. 与 OpenClaw / Hermes / SWE-Agent 的设计空间对照
 18. 20 条可迁移 Know-Why / Know-How
- 附录 A. 源码模块职责速查
- 附录 B. 证据与资料索引
- 附录 C. 术语表

1. 事件取证：到底泄露了什么？

2026-03-31 Source Map 事件：从发布到公开分析

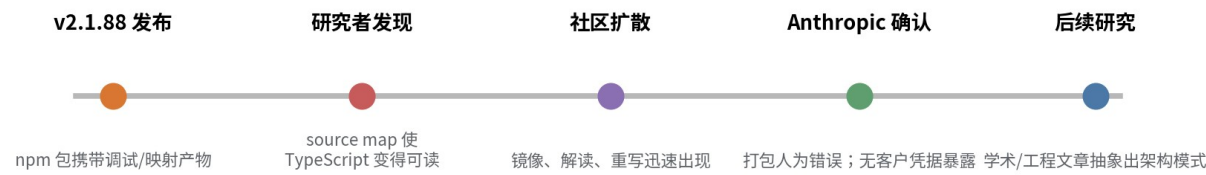


图 1-1 事件时间线（基于公开报道重绘）。

1.1 可以确认的事实

2026 年 3 月 31 日，Anthropic 的 Claude Code 发布包中意外包含了内部源码相关产物。Anthropic 对媒体确认：这是发布打包过程中的人为错误，而不是攻击者入侵；公司同时表示没有客户敏感数据或凭据被暴露。[A1]

多家媒体与后续学术分析把受影响版本指向 @anthropic-ai/claude-code v2.1.88。公开报道通常描述为一个约 60MB 的 JavaScript source map，使大约 1,900 个 TypeScript 文件、约 50-51 万行源码可读。不同分析者的文件/行数统计略有差异，因此本报告把这些数字视为近似量级，而非需要逐行一致的精确计量。[C1][C2][B1]

为什么 source map 这么关键 生产 JavaScript 往往被 bundle/minify。source map 的用途是把运行时位置映射回原始源码。若映射文件同时携带原始 sourcesContent，或者能指向可直接获取的原始源码产物，就可能把“不可读 bundle”恢复成接近开发态的 TypeScript。

证据：A1 Anthropic statement via BleepingComputer • B1 arXiv:2604.14228 • C1 WSJ • C2 Ars Technica

1.2 这不是“Claude 模型泄露”

被暴露/可推断	没有被暴露
Claude Code 客户端/agent harness 的大量实现细节	Claude 模型权重
系统 prompt / tool wiring / feature flags / UI & runtime 逻辑等线索	训练数据与训练管线全貌
部分未发布功能和内部实验的代码线索	客户 API key / customer credentials（Anthropic 明确否认）
安全边界与工具调度方式	Anthropic 全部后端基础设施源码

关键区别 “模型能力”与“Agent Harness 能力”是两层。Claude Code 的竞争力很大一部分来自 harness：如何给模型上下文、怎样让它调工具、如何限制风险、怎样恢复会话、怎样把扩展加载进来。泄露最有价值的是这层系统工程，而不是模型本身。

1.3 版权与“公开可见”不是一回事

源码被意外公开，不等于获得开源许可证。Anthropic 随后对 GitHub 上的镜像采取了版权下架行动。工程学习可以基于公开论文、官方文档和独立伪代码进行；复刻架构思想与复制专有表达是两回事。[C1]

2. 先纠正五个常见误解

误解 1 : Claude Code 的核心一定是复杂 Planner

源码级分析反而显示系统中心是一个 reactive while-loop / AsyncGenerator。复杂度主要在 loop 周围的 harness。

误解 2 : 权限弹窗就是安全

权限只是其中一层。真正架构包含 tool pre-filter、deny-first 规则、permission mode、classifier、sandbox、resume trust reset、hooks。

误解 3 : CLAUDE.md 是 system prompt

学术源码分析指出 v2.1.88 中 CLAUDE.md 被作为 user context 注入，因此它是概率性指导；确定性约束依赖 permission rules / sandbox。

误解 4 : Subagent 最大价值是并行

更重要的是隔离上下文。父 Agent 只拿 summary，避免把探索过程全部灌回主 context。

误解 5 : 泄露的隐藏功能等于未来一定发布

feature flag / dormant code 只能说明“做过或正在实验”。路线图判断必须与后续官方 docs/changelog 交叉验证。

阅读源码的正确姿势 优先寻找“稳定控制面”：入口、循环、状态、工具接口、权限决策、上下文构建、持久化；最后才看 UI、彩蛋和 feature flag。

3. 源码地图：从 50 万行看到 7 个系统组件

Claude Code：七组件高层结构（本报告重绘）

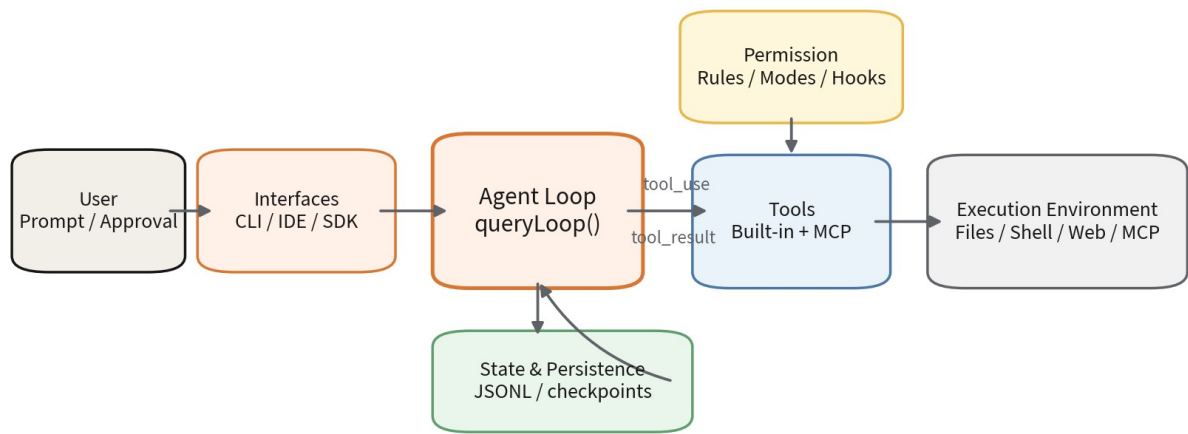


图 3-1 七组件结构。学术论文把所有入口最终归并到同一个 agent loop。

3.1 七件模型

组件	职责	源码级信号（v2.1.88 分析）
User	给目标、审批、审阅	prompt / permission interaction
Interfaces	CLI、headless、SDK、IDE/Desktop/ Browser	entrypoints、repl、cli handlers
Agent Loop	模型调用、tool dispatch、result collection	query.ts / queryLoop()
Permission System	allow/ask/deny、模式、classifier、hooks	permissions / useCanUseTool / yoloClassifier
Tools	Read/Edit/Bash/Agent/... + MCP	tools + services/tools
State & Persistence	session state、transcript、history、checkpoint	sessionStorage / history / file-history
Execution Environment	filesystem / shell / network / remote	sandbox / remote / MCP transports

证据：B1 § 3.2 / Appendix package map

3.2 为什么“七组件”比文件树更重要

大代码库最容易把阅读者淹没在文件名里。正确做法是先画数据流，再把文件挂到数据流节点。Claude Code 的典型一次动作可以写成：

```
User Prompt
-> Interface
-> Agent Loop assembles context
-> Model proposes tool_use
```



```
-> Permission gate decides  
-> Tool executes inside environment  
-> tool_result appended  
-> loop continues  
-> transcript persists
```

Know-how 遇到陌生 Agent 项目，先找“一个用户请求从哪里进入、在哪里调用模型、在哪里执行工具、结果如何回到模型、什么时候停止”。这五个问题通常能定位系统骨架。

4. 核心：Agentic Query Loop

核心 Agentic Query Loop：简单循环 + 重型 Harness

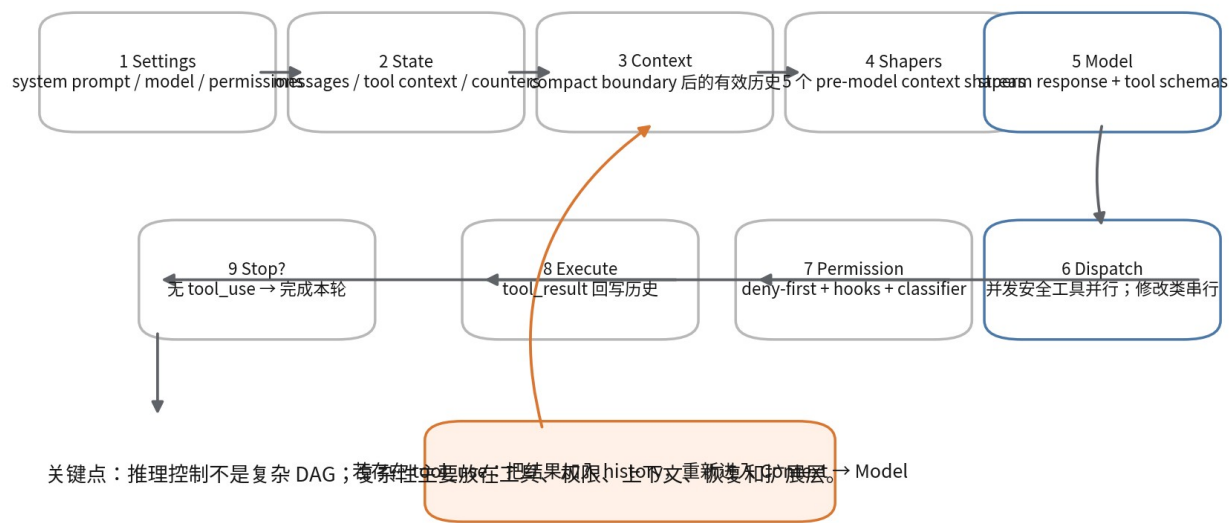


图 4-1 本报告根据公开源码分析重绘的 query pipeline。

4.1 queryLoop() 为什么是系统中心

公开学术分析将 query.ts 中的 queryLoop() 描述为 AsyncGenerator。它既维持单一控制流，又可以向 UI 持续 yield stream events；Interactive CLI 与 headless/SDK 最终共享的是 query() / queryLoop() 路径，而不是某个“万能 Engine 类”。[B1 § 3.4-4.1]

一个 turn 的关键顺序可抽象为：settings → mutable state → context assembly → five shapers → model streaming → tool dispatch → permission → execution/result → stop。只要 response 仍含 tool_use，结果就会作为 tool_result 回写历史并继续下一轮。

证据：B1 § 4.1

4.2 Minimal scaffolding, maximal operational harness

Know-Why 很多 Agent 框架把复杂度写进 planner、graph、state machine；Claude Code 更像是把“决策”交给模型，把工程复杂度投资到 operational harness：工具可达性、安全、上下文、恢复、并发、扩展与持久化。

这种选择的收益：控制流简单、延迟低、模型升级能自然吃到收益；代价：更依赖模型局部判断，没有内建的全局搜索/回溯保证，长任务需要用 verification、subagents、context management 兜底。

4.3 Reactive Loop 与 ReAct 的关系

源码级分析把它归为 ReAct 式 reactive architecture：模型产生 reasoning/action，harness 执行动作，结果进入下一轮。它不同于 LangGraph 式显式图，也不同于 tree search。

模式	Claude Code 选择	优点	主要代价
Reactive loop	是	简单、低延迟、适配强模型	不系统探索多条路径
Explicit DAG / graph	核心 loop 不是	可审计、可约束流程	框架复杂，可能限制模型
Tree search	不是默认	可比较多条轨迹	成本/延迟高
Orchestrator-workers	用于 subagents	并行/隔离	需要任务切分与汇总

5. Tool Runtime：并发、流式与结果回写

5.1 工具系统不是“一个 dict 映射函数”

教学版 Agent 常见：tool_name -> handler。生产版还要解决 schema assembly、feature gating、streaming、parallel safety、abort、progress、output order、result budget、permission integration 和 MCP 合并。

源码分析指出 Claude Code 有 StreamingToolExecutor 路径：工具调用可以在模型流式响应尚未结束时开始执行，从而降低多 tool-use 场景延迟。工具会被区分为“可并发安全”与“需要独占/串行”的类别；只读操作更适合并行，修改状态的 shell 类动作更倾向串行。[B1 § 4.2]

证据：B1 § 4.2

5.2 三个生产级细节

- Sibling abort**
当一个关键 Bash 子进程失败时，取消相关正在运行的兄弟进程，避免任务继续在错误前提上推进。
- Order preservation**
内部即使并行，返回给模型的 tool_result 顺序仍应与 tool_use 顺序稳定对应。
- Result budget**
单个工具输出必须有上限，否则一次 grep/log/test 就能吃掉大量 context。

5.3 工具设计的真正目标：为概率性调用者设计 affordance

传统 API 关注	Agent Tool 额外关注
参数是否合法	模型是否能正确选到它
返回值是否正确	返回是否包含下一步推理所需的最小上下文
接口稳定	schema 是否消耗过多 context
错误码	错误是否足以让模型自我修复
权限控制	调用前是否能被 policy/sandbox 拦截

Know-how 工具返回不要“把整个世界 dump 进来”。优先返回：结论、关键证据、可行动的错误、必要标识符；大对象留在文件/资源中，用引用按需读取。

6. Permission & Sandbox：真正的安全边界

安全不是一个 Permission Dialog，而是多层 Defense-in-Depth

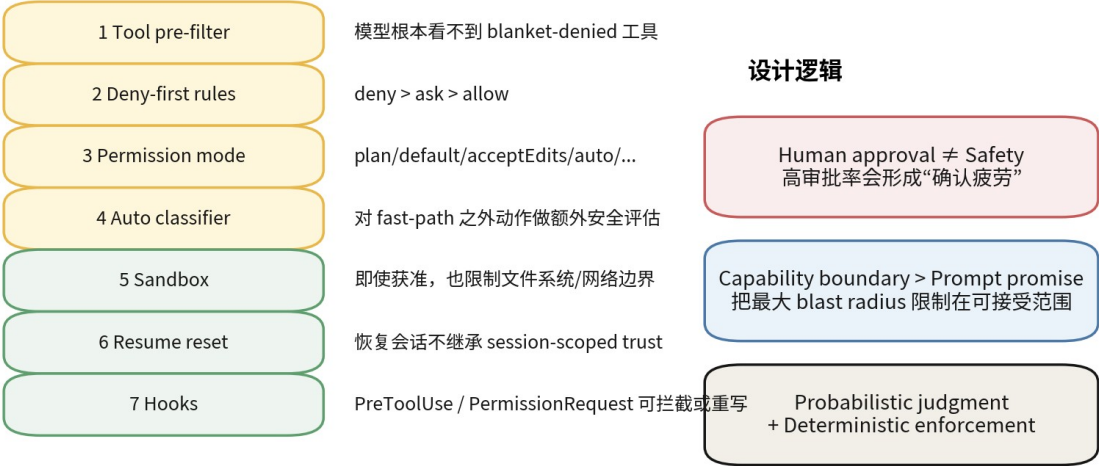


图 6-1 七层安全思路：任何一层都可以阻止动作。

6.1 七层 defense-in-depth

学术源码分析把 v2.1.88 的安全路径拆成七层：tool pre-filter、deny-first rule evaluation、permission mode、auto classifier、sandbox、resume 时不恢复 session trust、hook interception。[B1 § 3.5]

证据：B1 § 3.5 • A2/A3 official permissions docs

6.2 Deny-first 为什么比“最具体规则优先”更保守

源码分析指出 deny rule 总是优先，即使 allow 更具体。这样可以构造“不可被局部例外穿透”的安全地板。官方文档当前也明确说明权限规则按 deny → ask → allow 求值。

```
decision(request):
  if matches(DENY): return DENY
  if matches(ASK):  return ASK
  if matches(ALLOW): return ALLOW
  return behavior_of(permission_mode)
```

6.3 Permission Mode 是“信任梯度”，不是开/关

模式	源码快照/当前公开含义	适合场景
plan	研究/规划，限制修改	陌生 repo、敏感任务
default	读优先，风险动作询问	高监督
acceptEdits	工作区编辑更自动	日常开发
auto	后台安全分类器替代大量 prompt	长任务，研究预览
dontAsk	未预批准动作自动拒绝	CI/locked-down automation

bypassPermissions	跳过绝大部分交互权限	仅隔离容器/VM
bubble	源码分析中的内部 subagent escalation 模式	内部机制，不是用户模式

当前官方文档把 auto mode 标为 research preview，并强调它不能保证安全。官方描述的关键设计是：先应用显式 allow/deny，工作区只读/编辑可走 fast path，其余动作交给单独 classifier；classifier 还会读取用户对话里声明的边界。[A2]

6.4 Sandbox : 从“判断命令”升级到“约束能力边界”

Know-Why 逐次问“这个命令安全吗？”是概率判断；sandbox 直接定义“即使 Agent 判断错，它最多能碰哪里”。这把风险从模型判断问题转化为 capability containment 问题。

工程上最稳妥的自治组合通常是：narrow tool set + deny rules + sandbox + branch/worktree isolation + verification，而不是单独开启 bypass。

7. Context & Memory ： Claude Code 的第一资源约束

Context 是绑定资源：五级渐进式压缩 + 多处节流



旁路节流：CLAUDE.md 懒加载 · ToolSearch 延迟 schema · Subagent summary-only · per-result budget

核心原则：先用最小破坏策略；只有在压力继续上升时才升级压缩级别。

图 7-1 五级渐进式 Context Reduction。

7.1 Context as bottleneck

源码分析明确把 context window 视为 binding resource constraint。这个判断解释了很多看似无关的实现：CLAUDE.md 懒加载、ToolSearch 延迟 schema、subagent summary-only、per-tool-result budget、五级 compaction。它们其实都是同一条资源治理策略的不同落点。[B1 § 3.6, § 7]

证据：B1 § 3.6 / § 7

7.2 CLAUDE.md 四层 memory hierarchy

层级	典型位置	用途
Managed memory	系统/企业管理位置	组织级政策
User memory	~/.claude/CLAUDE.md	个人全局规则
Project memory	CLAUDE.md / .claude/CLAUDE.md / rules	团队可版本化规则
Local memory	CLAUDE.local.md	gitignored 私有项目规则

源码分析特别指出：CLAUDE.md 作为 user context 进入，而不是 system prompt。因此它是一种“指导层”；真正硬约束仍依赖 permissions/sandbox。这是非常值得迁移的分层：

Guidance (probabilistic): CLAUDE.md / Skills / prompts
Enforcement (deterministic-ish): rules / sandbox / hooks / OS boundaries

7.3 五级 Compaction Pipeline

层	策略	破坏性	直觉
1	Budget reduction	低	先限制单个大工具结果

2	Snip	低-中	裁掉旧历史的低价值部分
3	Microcompact	中	细粒度/cache-aware 压缩
4	Context collapse	中-高	读取时构造虚拟投影
5	Auto-compact	高	让模型生成会话摘要

关键不是“有 /compact”，而是 lazy degradation：先用最便宜、最不破坏信息的策略；只有压力继续上升才升级。压缩之后，运行态 attachments（例如 plan、skills、async agents）还要重新注入，因为历史消息可能被压缩，但真实 runtime state 仍存在。

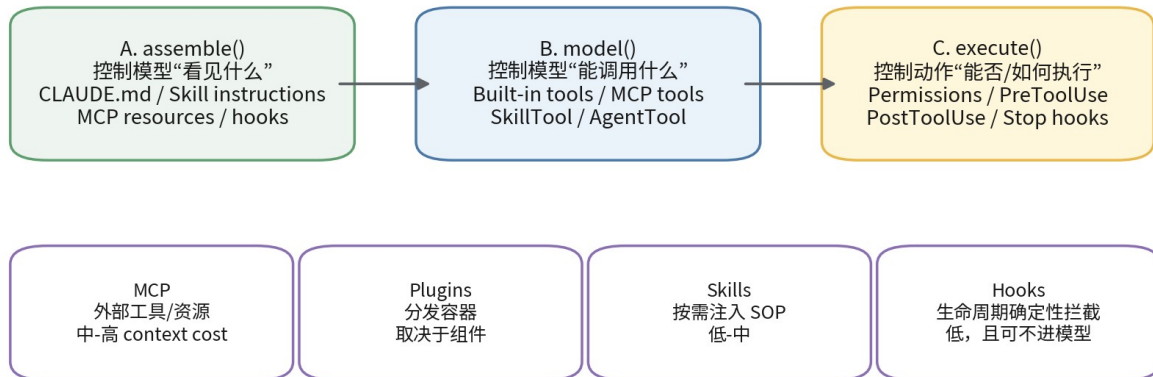
7.4 为什么透明文件式 Memory 是一种产品哲学

透明文件式 memory 的优点是可读、可编辑、可版本化；缺点是检索粒度与自动整理能力弱于数据库/向量检索。Claude Code 的选择体现的是“inspectability over opaque retrieval”。

Know-how 做自己的 Agent Memory 时，先区分三类：Policy（必须遵循）、Project Context（可推断/可版本化）、Experience（会增长、需整理）。不要把三类全部塞进一个长期 prompt。

8. Extensibility : MCP / Plugins / Skills / Hooks

扩展机制不是一个 API：它们插入 Agent Loop 的不同位置



工程判断：需要“理解”的放 context；需要“必然发生”的放 hook；需要外部能力的放 MCP/Plugin。

图 8-1 四类扩展分别改变“看见什么 / 能调用什么 / 怎么执行”。

8.1 为什么不用一个统一 Extension API？

Claude Code 的扩展面选择“多机制组合”，因为不同扩展需要进入 agent loop 的位置不同，context cost 也不同。MCP 是外部能力协议；Plugin 是分发容器；Skill 是按需知识/流程注入；Hook 是生命周期确定性逻辑。[B1 § 6][A4][A5]

证据：B1 § 6 • A4 official plugin docs • A5 official MCP docs

8.2 MCP：扩展 action surface

源码分析描述 MCP server 从多 scope 合并，并把 server tools 变成与内建工具同池的 MCPTool。官方文档当前推荐 HTTP 作为远程 MCP 的主要连接方式。

Know-how 如果外部系统已有成熟 CLI，CLI 往往 context 更省；如果需要结构化 schema、权限与跨团队封装，MCP 更适合。

8.3 Plugin：把多个扩展面打包成一个产品单元

当前官方 Plugin reference 显示插件可包含 skills、agents、hooks、MCP、LSP、monitors 等。源码分析同样把 plugin loader 视为“分发与组件注册层”，而不是一个新的执行引擎。

8.4 Skill：程序化 SOP，而不是大 Prompt 仓库

Skill 最值得学习的是 progressive disclosure：平时只暴露 metadata/description；命中任务时才把完整 SKILL.md 注入。这样可以把“每个任务都需要的 invariant”留在 CLAUDE.md，把“某一类任务的 SOP”放 Skill。

8.5 Hook : 把概率性提醒升级成确定性事件处理

需求	更适合放哪里	原因
“这个项目使用 pnpm”	CLAUDE.md	全局知识
“发布必须按 12 步 SOP”	Skill	任务型流程
“每次 Edit 后自动 formatter”	Hook	必须发生
“访问 Jira / DB”	MCP	外部能力
“打包整套团队扩展”	Plugin	分发/版本化

9. Subagents / Worktrees / Agent Teams

Subagent 的第一价值不是并行，而是 Context Isolation

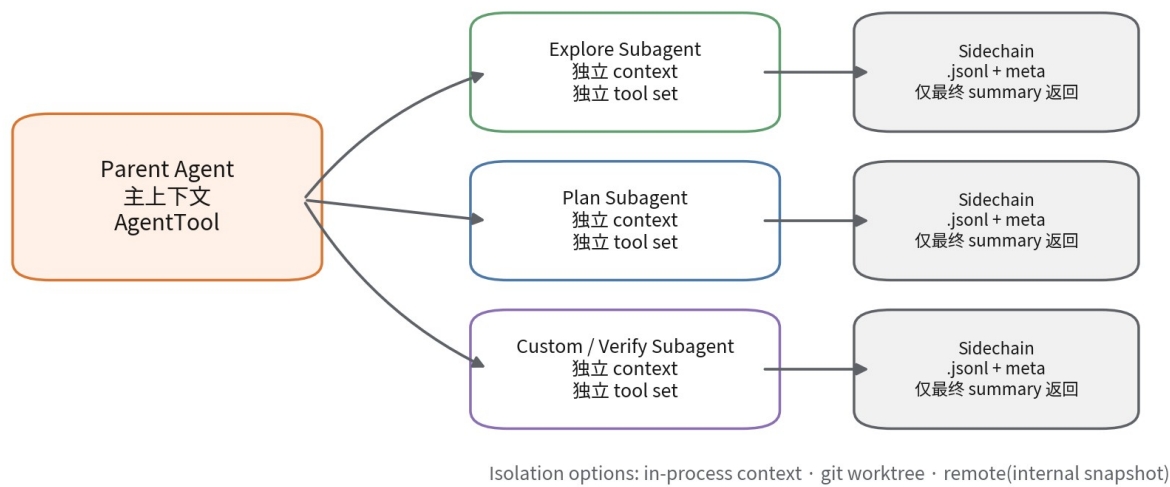


图 9-1 子 Agent：sidechain 保存完整历史，父上下文只接收最终 summary。

9.1 AgentTool：真正的委派边界

源码分析把 AgentTool 视为 subagent delegation 入口。Built-in 类型包含 Explore、Plan、general-purpose 以及若干专用 agent；自定义 agent 可以拥有自己的 tools、model、permission mode、MCP、hooks、skills、memory scope、isolation 等。

关键：默认 subagent 不继承父对话的全部历史，因此委派 prompt 必须 self-contained；代价是需要更认真地写任务包，收益是 context 不爆炸。

证据：B1 § 8

9.2 三种隔离维度

维度	实现	解决的问题
Context isolation	独立 conversation window	探索噪声不污染父上下文
Filesystem isolation	git worktree	并行写不会互相覆盖
Lifecycle isolation	sync / background	长任务与主线程解耦

9.3 Worktree 为什么是 Coding Agent 的甜点位隔离

容器隔离更强，但基础设施更重；仅 context isolation 又会共享 working tree。git worktree 恰好解决“并行编码时最常见的冲突”：同一 repo，不同 working tree/branch，不需要完整容器编排。

9.4 Agent Teams : 何时值得付 7× token 级成本

源码分析/官方材料都提醒多 Agent 协作成本显著。适合：多个独立假设并行调查、前后端/测试拆分、reviewer 与 implementer 分离；不适合：本质串行的小任务。

Know-Why Subagent 的 ROI = 节省主 context + 并行获得独立证据 - 委派/汇总成本。只有当独立子问题足够大时，才应该 spawn。

10. Session Persistence / Compact / Resume / Rewind

Session : Live Context 与 Durable Transcript 分离

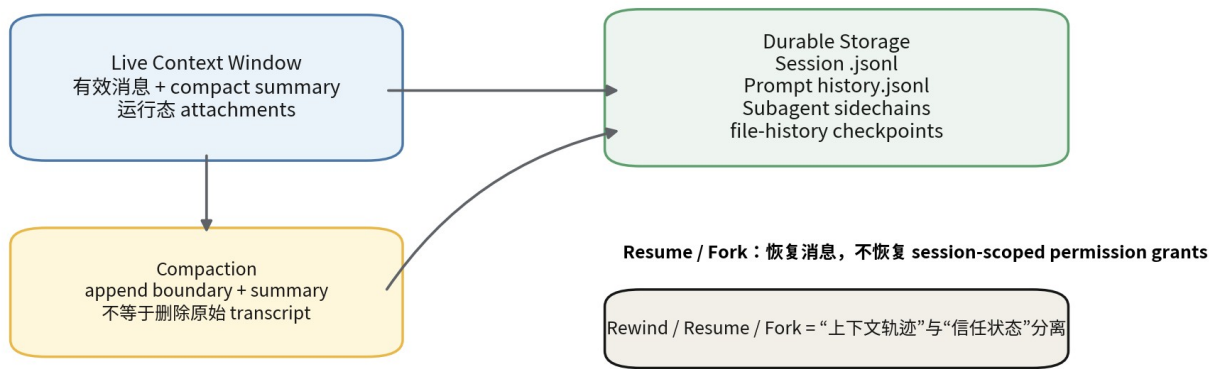


图 10-1 Live Context 只是 transcript 的一个“当前视图”。

10.1 Mostly append-only JSONL

源码分析显示 session transcript 以 mostly append-only JSONL 保存；compaction 通常不是去改写/删除旧行，而是追加 compact boundary 和 summary 事件，再在读取时重建有效消息链。[B1 §9]

证据：B1 §9

10.2 三类持久化通道

通道	保存什么	价值
Session transcript	user/assistant/tool/attachment/ compact metadata	审计、resume、fork
Global prompt history	用户输入历史	终端历史导航
Subagent sidechain	每个子 agent 的 .jsonl + meta	调试完整，父 context 不膨胀

10.3 Resume 恢复 Conversation，不恢复 Trust

这是非常重要的安全设计。源码分析指出 resume/fork 会重放消息，但 session-scoped permissions 不会持久化继承。理由是“信任决定属于当时的会话环境”，恢复时 repo、配置或任务语境可能已经变化。

Know-how 持久化 Agent 时，不要把“历史状态”和“授权状态”混成一份快照。历史可恢复，权限应重新计算。

10.4 Checkpoint 不等于 Git

Claude Code 的 file-history checkpoint 适合短期 rewind；长期可审计版本仍应依赖 Git。这也是 “Agent runtime recovery” 和 “software configuration management” 职责分离。

11. 隐藏功能与路线图线索：什么能信，什么不能信

重要 这一章只讨论公开媒体对 v2.1.88 源码快照的报道。Feature flag、unused module、内部命名只说明“做过/试过/可能准备过”，不等于正式发布，也不保证最终设计一致。

11.1 KAIROS : Persistent / Proactive Agent 的方向

Ars Technica、The Verge 等报道称源码中出现 KAIROS：一个可能在终端关闭后仍持续运行、通过周期性 tick 判断是否需要主动行动的 background agent 方向。[C2][C3]

真正值得学习的不是名字，而是产品方向：Coding Agent 从“用户 prompt 驱动”走向“事件/时间驱动”，这会把 runtime 从 CLI session 升级成 daemon/service。随之必须重新设计长期 memory、notification、permission freshness、resource quotas 和 audit。

11.2 AutoDream : Memory consolidation 而不是无限 append

DeepLearning.AI、Ars 与 WSJ 都报道了类似“dreaming/AutoDream”的记忆整理线索：在空闲或会话结束时，让 Agent 回顾记忆并做 consolidation。[C4][C2][C1]

Know-Why 长期 Agent 的问题不是“会不会记”，而是“会不会忘错、记错、重复记、越记越大”。Memory consolidation 是把 append-only experience 变成可维护长期状态的必经层。

11.3 Buddy : 产品 engagement 与核心架构无关

The Verge/Ars 报道源码里有 Tamagotchi 风格的 Buddy/pet 线索。它说明 Claude Code 的客户端不仅是 runtime，也有 engagement/UI experiment，但对理解 Agent 核心机制价值较低。[C2][C3]

11.4 Undercover : 高争议、应谨慎阅读

WSJ、Ars 与社区文章报道过“Undercover”相关内部模式线索，涉及发布代码时减少/隐藏 AI attribution。由于这不是官方公开功能，而且语境可能是内部实验，本报告只将它列为低于核心架构的路线图/策略线索，不做产品行为断言。[C1][C2]

11.5 泄露后哪些东西已进入公开产品？

截至本报告研究截止日，官方文档已经公开并强化了 Auto mode、Worktree、Agent View/Teams、Plugins、Hooks、MCP、LSP、Ultrareview 等一整套 Agent Runtime 能力；而本次检索没有在官方 docs map/changelog 中找到 KAIROS、Buddy、AutoDream 作为正式公开功能的文档入口。应继续把它们视为“源码快照线索”，而不是当前操作指南。

证据：A6 official changelog/whats-new • C1/C2/C3/C4 media reports

12. 从源码抽象出的 13 个设计原则

原则	工程解释
1 Deny-first + human escalation	未知风险动作不默认放行；人工审批是升级路径，不是唯一安全层。
2 Graduated trust spectrum	权限不是二元开关，而是可随经验迁移的信任梯度。
3 Defense in depth	规则、classifier、sandbox、hooks、protected paths 叠加。
4 Externalized programmable policy	policy 可配置、可版本化，并可通过 hooks 编程。
5 Context as scarce resource	context 被当作 CPU/RAM 一样治理，而不是无限历史。
6 Append-oriented durable state	transcript 以事件形式持久化，live context 只是投影。
7 Minimal scaffolding, maximal harness	少做硬编码 planner，多做可靠 runtime。
8 Values over rigid rules	模型负责语境判断；关键边界仍由确定性机制兜底。
9 Composable extensibility	MCP/Skill/Hook/Plugin 分别在不同 injection point 扩展。
10 Reversibility-weighted risk	只读/可逆动作与不可逆动作使用不同监督成本。
11 Transparent file-based config/memory	让人能直接检查和版本化 Agent 的规则与记忆。
12 Isolated subagent boundaries	子任务的上下文、工具、文件系统尽可能隔离。
13 Graceful recovery	token cap、tool failure、compact、resume 都优先恢复而非 fail hard。

证据：B1 § 2.2

最重要的抽象 Claude Code 的架构并不是“13 条原则的堆砌”。它们最后收敛为三个系统主题：有限资源治理（Context）、风险治理（Permission/Isolation）、复杂度治理（Simple loop + composable extensions）。

13. 泄露事件本身暴露的工程问题

13.1 Source map 是 Release Artifact，不是开发垃圾

这次事件最大的通用教训是：安全扫描不能只看 secrets 和 dependency CVE，还要看“最终发布包里到底有什么”。source map、debug symbols、test fixtures、internal docs、private endpoint names 都属于 artifact hygiene。

13.2 防线应该放在 publish gate，而不是依赖某个人记得 .npmignore

```
build
-> package manifest allowlist
-> inspect tarball contents
-> fail if *.map / source archives / private files present
-> secret scan
-> size anomaly check
-> publish
-> post-publish canary verification
```

Know-how 最稳的发布策略是 allowlist：明确 files 字段或打包清单，而不是不断给 .npmignore 增加 deny pattern。

13.3 AI-assisted development 需要“artifact-level verifier”

2026 年 VibeGuard 等研究直接把这次事件当作 packaging blind spot：传统代码质量检查可能全部通过，但最终 npm tarball 仍能泄露内部源码。Agent 写代码越快，release pipeline 的机械验证越重要。

13.4 泄露不是“安全漏洞”≠没有安全后果

Anthropic 把事件定性为 packaging issue，而非 intrusion，这在事故原因上成立；但源码暴露仍会提高攻击者分析 guardrail、解析器差异和未发布路径的效率，同时也给竞争对手更完整的 harness 蓝图。因此 incident taxonomy 与 impact taxonomy 应分开。

14. 复刻一个 Mini Claude Code：最小可行 Runtime

14.1 第一阶段：只实现 loop

```
async def agent_loop(messages, tools):
    while True:
        response = await model(messages, tools)
        if not response.tool_calls:
            return response.text
        results = await execute_tools(response.tool_calls)
        messages += [response, *results]
```

这对应教学版 s01：证明“模型 → 工具 → 结果 → 模型”的闭环。

14.2 第二阶段：Tool Runtime

模块	必须解决
Tool interface	schema / validation / description
Dispatcher	name → executor
Concurrency	read parallel / write serialized
Cancellation	abort siblings on critical failure
Result normalization	稳定 tool_result 结构
Budget	输出限额 / reference large artifacts

14.3 第三阶段：Policy Gate

```
request
-> pre_filter(tool visibility)
-> pre_tool_hook
-> deny/ask/allow rules
-> permission mode
-> optional classifier
-> sandbox
-> executor
```

14.4 第四阶段：Context Manager

不要一开始就做 vector database。先实现：token budget → tool result cap → history snip → summary compact → persistent project rules。等真实使用证明需要，再增加 retrieval。

14.5 第五阶段：Delegation + Persistence

Subagent 必须拥有独立 message list；父 agent 只收 summary。Session 使用 append-only JSONL，恢复时重新计算 permissions。Worktree 用于并行写任务。

14.6 推荐目录

```
agent_runtime/  
  loop.py  
  context.py  
  model.py  
  tools/  
  permissions/  
  sandbox/  
  hooks/  
  memory/  
  agents/  
  persistence/  
  mcp/  
  cli/  
  tests/
```

成功标准 你的 Mini Runtime 不应以“功能数量”衡量，而应以：能否稳定复现任务、能否验证结果、能否限制最大风险、能否恢复、能否观测每次决策来衡量。

15. 从 s01 → s10：可累积的 Agent Runtime 教学路线

阶段	核心能力	Claude Code 对应层	你真正学到什么
s01	Agent Loop	queryLoop	闭环与 stop condition
s02	Multi-tool	tools / dispatch	schema + execution
s03	Permission	permissions	policy gate
s04	Context	query/context	有限工作内存治理
s05	Memory	CLAUDE.md / memdir	持久指导 vs 经验
s06	Skills	skills / SkillTool	progressive disclosure
s07	Hooks	hook lifecycle	deterministic automation
s08	Subagents	AgentTool	context isolation
s09	Sandbox / Worktree	execution boundary	blast radius containment
s10	Multi-agent / Persistence	teams / JSONL	协调、恢复、审计

Know-how 每一阶段都保留上一阶段的可运行代码，再只加入一个新系统约束。这样“复杂 Agent”不会变成黑盒，而是一条可以逐层解释的演化链。

16. 源码阅读方法：如何分析一个 50 万行 Agent 产品

16.1 不要从 main.tsx 顺序读到最后

更高效的方法是“行为追踪 + 边界定位”：先选一个真实任务，例如 fix failing test，然后追踪一个完整 turn。

- 1. 找入口：prompt 如何进入 runtime？
- 2. 找 model call：哪个函数真正调用模型？
- 3. 找 tool schema assembly：模型看到哪些工具？
- 4. 找 permission gate：tool_use 在哪里被批准/拒绝？
- 5. 找 executor：命令/文件操作在哪里发生？
- 6. 找 tool_result：结果如何回到 conversation？
- 7. 找 stop / recovery：什么时候结束、失败如何继续？
- 8. 找 persistence：哪些状态落盘，哪些只在内存？

16.2 四类“高价值文件”优先级

优先级	类型	为什么
P0	loop / context / permission / persistence	定义系统语义
P1	tool interface / orchestration / sandbox	决定行动能力和风险
P2	skills/hooks/MCP/agents loaders	决定可扩展性
P3	UI / commands / themes / experiments	产品体验或功能入口

16.3 用 invariants 读源码，而不是只记函数名

- Tool result 必须与 tool_use 可稳定关联。
- Deny 不应被更窄 Allow 绕过。
- Resume 不自动恢复旧 trust。
- Compaction 不应丢失仍然有效的 runtime state。
- Subagent 完整 transcript 不应默认进入父 context。

Know-Why 函数名会改，feature flag 会删，文件会重构；invariant 才是可迁移的架构知识。

17. 与 OpenClaw / Hermes / SWE-Agent 的设计空间对照

设计问题	Claude Code	OpenClaw / Hermes / SWE-Agent 类替代
核心控制流	单 CLI reactive loop	Gateway runtime / 单进程多入口 / benchmark-oriented loop
安全	per-action deny-first + classifier + sandbox	perimeter IAM / per-action approval / container-first
Context	五级压缩 + lazy loading	structured long-term memory / simpler compaction
扩展	MCP + Plugin + Skill + Hook	gateway capability registry / plugin memory backends
多 Agent	task delegation + worktree	gateway routing / container workers
持久化	append-oriented JSONL	DB / gateway state / benchmark ephemeral

这里没有“谁绝对更好”。部署上下文不同，会改变最优边界：CLI agent 假设本地开发机是核心执行环境，因此需要细粒度 per-action policy；云端 gateway 更常把安全放到 perimeter identity / tenant isolation。

18. 20 条可迁移 Know-Why / Know-How

01. Agent Loop 越简单，越要把工程投资放到 harness。
02. Context window 是运行时预算，不是聊天记录仓库。
03. 先减少无用信息，再做摘要；摘要应该是最后级别的压缩。
04. Tool schema 也是 context 成本，按需加载优于全量常驻。
05. Tool output 要预算化；大结果应留在外部资源。
06. 人工审批不能作为唯一安全层，用户会形成确认疲劳。
07. Deny-first 可以形成不可被局部例外穿透的安全地板。
08. Sandbox 约束“最大能做什么”，比 prompt 约束“最好别做什么”更可靠。
09. Guidance 与 Enforcement 应分层：CLAUDE.md/Skill != Permission Rule。
10. Hook 适合 deterministic invariant；Prompt 适合 contextual judgment。
11. Skill 是可按需加载的 SOP，不是所有知识都应该进 system prompt。
12. Plugin 是分发单元，不是新的 agent loop。
13. MCP 是 capability protocol；不要把它误当 memory system。
14. Subagent 首要价值是 context isolation，其次才是并行。
15. 并行写代码必须配 filesystem isolation，worktree 是低成本方案。
16. 父 Agent 不应继承子 Agent 全部探索历史；summary-only 是有效压缩边界。
17. Conversation state 可以恢复，trust state 应重新建立。
18. Append-only transcript 让 compaction 成为“视图变化”，而不是数据销毁。
19. Verification 是 Agent 的外部 ground truth；没有 verifier 的“完成”只是模型自评。
20. 发布安全要检查最终 artifact，而不是只检查源码仓库。

附录 A. 源码模块职责速查（v2.1.88 公开分析快照）

注意 下面的文件/目录映射来自学术论文对公开源码快照的重建分析；论文自身明确标注该附录为 reconstructed analysis，而非 Anthropic 官方架构文档。

区域	代表文件/目录	职责
Entry & Startup	main.tsx / replLauncher / entrypoints / cli	启动、模式分发、交互/SDK/headless 入口
Core Loop	query.ts / query/ / QueryEngine / context.ts	turn execution、context assembly、streaming loop
Tools	Tool.ts / tools/ / services/tools/	tool interface、具体工具、执行与 orchestration
Safety	utils/permissions / types/permissions / useCanUseTool	规则、模式、classifier、permission handler
Extensions	plugins / skills / hooks	组件加载、SkillTool、hook lifecycle
Context & Memory	services/compact / memdir / claudemd	五级 compaction、auto memory、CLAUDE.md hierarchy
Persistence	history / sessionStorage	prompt history、session JSONL、sidechains、file history
Integration	services/mcp / LSP / remote / coordinator	外部工具、代码智能、远端/多 Agent 协调
UI & optional	components / screens / outputStyles / buddy	终端 UI 与实验功能

证据：B1 Appendix Figure 9 / package structure

附录 B. 证据与资料索引

ID	来源	标题	URL
A1	Anthropic statement quoted by BleepingComputer	Claude Code source code accidentally leaked in NPM package	https://www.bleepingcomputer.com/news/artificial-intelligence/claude-code-source-code-accidentally-leaked-in-npm-package/
A2	Claude Code Docs	Choose a permission mode	https://code.claude.com/docs/en/permission-modes
A3	Claude Code Docs	Configure permissions	https://code.claude.com/docs/en/permissions
A4	Claude Code Docs	Plugins / Plugins reference	https://code.claude.com/docs/en/plugins
A5	Claude Code Docs	Connect Claude Code to tools via MCP	https://code.claude.com/docs/en/mcp
A6	Claude Code Docs	Changelog / What's new	https://code.claude.com/docs/en/changelog
B1	Jiacheng Liu et al., arXiv v2, 2026-07-02	Dive into Claude Code: The Design Space of Today's and Future AI Agent Systems	https://arxiv.org/abs/2604.14228
C1	The Wall Street Journal	Anthropic Races to Contain Leak of Code Behind Claude AI Agent	https://www.wsj.com/tech/ai/anthropic-races-to-contain-leak-of-code-behind-claude-ai-agent-4bc5acc7
C2	Ars Technica	Here's what that Claude Code source leak reveals about Anthropic's plans	https://arstechnica.com/ai/2026/04/heres-what-that-claude-code-source-leak-reveals-about-anthropics-plans/
C3	The Verge	Claude Code leak exposes a Tamagotchi-style pet and an always-on agent	https://www.theverge.com/ai-artificial-intelligence/904776/anthropic-claude-source-code-leak
C4	DeepLearning.AI - The Batch	Claude Code's Source Code Leaked, Exposing Potential Future Features Kairos and autoDream	https://www.deeplearning.ai/the-batch/claude-codes-source-code-leaked-exposing-potential-future-features-kairos-and-autodream/
C5	The Information / Bloomberg / Business Insider 等	事件确认与商业/安全影响交叉报道	见公开报道
D1	Educational reverse-analysis project	claude-code-from-source (声明不包含原始源码, 使用原创伪代码)	https://github.com/alejandrobalderas/claude-code-from-source
R1	arXiv:2608.11095	Why Does CLAUDE.md Keep	https://arxiv.org/abs/2608.11095

		Growing? Catastrophic Remembering in Agentic Coding	
R2	arXiv:2604.01052	VibeGuard: A Security Gate Framework for AI-Generated Code	https://arxiv.org/abs/2604.01052

研究方法说明：当同一细节只在社区文章出现、而学术论文或主流媒体没有交叉支撑时，本报告不把它作为核心架构事实。涉及未发布功能时，均使用“线索/报道/快照”措辞。

附录 C. 术语表

术语	在本报告中的含义
Agent Harness	围绕模型的工具、上下文、安全、执行、恢复、持久化等运行时系统
Agent Loop	模型调用 → 工具执行 → 结果回写 → 再调用的循环
Context Shaper	在模型调用前对上下文进行裁剪/压缩/注入的步骤
Compaction	将长对话压缩为更小有效视图的策略
Permission Mode	定义未命中显式规则时的默认授权姿态
Deny-first	规则冲突时 deny 优先
Sandbox	OS/进程级文件系统或网络能力边界
Skill	按需加载的领域流程/说明包
Hook	生命周期事件上的确定性处理器
MCP	Model Context Protocol，外部 tool/resource 集成协议
Subagent	拥有独立上下文（及可选隔离环境）的子任务执行体
Worktree	Git 的并行工作区机制，用于文件系统隔离
Sidechain	子 Agent 独立 transcript，不默认进入父上下文
Append-only JSONL	以追加事件为主的会话持久化格式
Source map	将 bundle/minified JS 映射回源码位置/内容的调试产物

END

最值得带走的一句话：

Claude Code 的“智能”来自模型，Claude Code 的“可用性”来自 Harness。